

Klassiker 14a: Selection Sort

Zum Foliensatz 14a — Sortieralgorithmen · SoSe 2026

Dorian Zwanzig

Die Aufgabe

Implementiere **Selection Sort** aus der Ein-Satz-Spezifikation: „Finde das Minimum im unsortierten Rest und tausche es nach vorn — wiederhole.“ Sortiere damit die vier Sale-Tagespreise [109.00, 135.50, 89.95, 59.95] **aufsteigend** — ohne `sorted()` und ohne `list.sort()`. Gib die Liste vor dem Sortieren, **nach jeder Runde** und am Ende aus.

Erwartetes Verhalten

```
Vorher:      [109.0, 135.5, 89.95, 59.95]
Runde 1:     [59.95, 135.5, 89.95, 109.0]
Runde 2:     [59.95, 89.95, 135.5, 109.0]
Runde 3:     [59.95, 89.95, 109.0, 135.5]
Sortiert:    [59.95, 89.95, 109.0, 135.5]
```

Erlaubte Bausteine

Alles bis Notebook 14a — mehr brauchst du nicht: Listen und Index-Zugriff `liste[i]` · `len()` · `for i in range(...)` (auch verschachtelt) · Vergleiche (`<`, `>`) · `if` · Tupel-Tausch `a, b = b, a` · f-Strings

Gestufte Hinweise

Erst selbst probieren — dann Hinweis für Hinweis aufdecken.

Hinweis 1 (Ansatz): Denke in **Zielpositionen**: Runde für Runde wird die Position `i = 0, 1, 2, ...` endgültig gefüllt. Links von `i` ist alles fertig sortiert (die Invariante!), rechts liegt der unsortierte Rest — dort suchst du das Minimum (das „Minimum finden“-Muster aus Foliensatz 10).

Hinweis 2 (Struktur): Pseudocode:

```
fuer jede zielposition i von 0 bis n-2:
    min_idx = i
    fuer j von i+1 bis n-1:
        wenn liste[j] < liste[min_idx]: min_idx = j
    tausche liste[i] und liste[min_idx]
    liste ausgeben
```

Hinweis 3 (der Kniff): Merke dir in der inneren Schleife den **Index** des bisherigen Minimums (`min_idx`), nicht seinen Wert — zum Tauschen musst du wissen, **wo** das Minimum steht. Und tausche erst **nach** der inneren Schleife: Wer schon während der Suche tauscht, verschiebt Elemente mitten im Durchlauf und verliert die Position des echten Minimums. Ein einziger Tausch pro Runde genügt.

Bonus

Vergleiche zählen: Zähle mit einer Variablen jeden Vergleich der inneren Schleife und gib die Zahl am Ende aus. Vergleiche sie mit der Formel aus der Vorlesung: $n(n-1)/2$ — bei $n = 4$ also 6. Teste auch eine bereits sortierte Liste: Selection Sort vergleicht **immer** gleich oft.

Musterlösung nach der Vorlesung: `Praesentationen_src/14a_Sortieralgorithmen/90_klassiker_selection_sort.py` · Dieselbe Aufgabe kommt bewusst als Übung im Notebook 14a wieder — Wiederholung erwünscht.